

PhonePe Pulse Data Analysis Project Documentation

Introduction

Digital payments have become an essential part of everyday transactions in India. Platforms like PhonePe process millions of transactions across different regions.

In this project, I worked on analyzing PhonePe Pulse data to understand transaction patterns, user growth, and regional performance across states and districts.

The goal was to take raw data, process it, and present meaningful insights through an interactive dashboard.

Objective of the Project

The main objectives of this project are:

- To analyze transaction volume and value across different regions
- To identify top-performing states and districts
- To study user growth and engagement trends
- To analyze insurance-related data
- To build an interactive dashboard for better visualization

In simple terms, this project answers:

Where and how are digital payments growing in India?

Data Source

The dataset used in this project is taken from the PhonePe Pulse GitHub repository.

Dataset Details:

Format: JSON

Contains: Transaction data, User data, Insurance data

Organized by: Year, Quarter, State, District

Since the data is nested and semi-structured, it required preprocessing before analysis.

Tools and Technologies Used

Programming: Python (Data extraction and processing), **Pandas & NumPy** (Data manipulation)

Database: MySQL (Structured data storage)

Visualization: Streamlit (Interactive dashboard), **Plotly** (Charts and graphs)

Version Control: Git & GitHub

Project Architecture

The project follows a structured pipeline:



Explanation:

- **JSON data is extracted using Python**
- **Semi-structured data is converted into tabular format for organizing the data**
- **Stored in MySQL for efficient querying**
- **Streamlit connects to MySQL to display insights through visualization**

Data Processing (ETL Pipeline)

Step 1: Data Extraction

JSON files are Loaded using Python

Extracted fields like: State, District, Transaction count, Transaction amount

Step 2: Data Transformation

Converted nested JSON into structured tables for easier data fetch

Standardized state and district names for clear processing

Handled missing values and inconsistencies by proper inspection of JSON files

Step 3: Data Loading

Created MySQL database

Stored data into tables: Aggregated Transaction, Map Transaction, User Data

This ensures efficient querying and scalability.

Sample Queries (Proof of Work)

Below are some SQL queries used in the project:

a. Top States by Transaction Amount

```
SELECT state, SUM(transaction_amount) AS total_amount  
FROM aggregated_transaction  
GROUP BY state  
ORDER BY total_amount DESC;
```

b. District-wise Transaction Analysis

```
SELECT district, SUM(transaction_count) AS total_transactions  
FROM map_transaction  
GROUP BY district  
ORDER BY total_transactions DESC;
```

c. User Growth Over Time

```
SELECT year, quarter, SUM(user_count) AS total_users
```

```
FROM user_data
```

```
GROUP BY year, quarter
```

```
ORDER BY year, quarter;
```

These queries help extract meaningful insights from the data.

Dashboard Development

The dashboard was built using Streamlit and connected to the MySQL database.

Features

- **Interactive filters: Year, Quarter, State**
- **Real-time data visualization**
- **Easy navigation for users**

Data Visualization

The project uses Plotly to create:

- Bar charts – State & district comparison
- Line charts – Growth trends
- Maps for Regional visualization

These visualizations make it easier to understand patterns in the data.

Detailed Processing – step by step

1. Analysis Process

In this project, I followed a step-by-step approach to convert raw data into meaningful insights.

First, I collected the data from the PhonePe Pulse GitHub repository. The data was available in JSON format, which is semi-structured and not directly suitable for analysis.

Next, I used Python to extract the required information such as state, district, transaction count, and transaction amount. Since the data was nested, I converted it into a structured tabular format using Pandas.

After cleaning and transforming the data, I stored it in a MySQL database. This made it easier to query and analyze the data efficiently.

Finally, I connected the database to a Streamlit application, where I built an interactive dashboard to display the results.

In simple terms:

I converted semi-structured JSON data into a cleaned one, stored it, and visualized it.

2. Insights Derived from the Data

After analyzing the data, several useful insights were observed:

- Some states consistently show higher transaction volumes compared to others
- Digital payment usage is increasing steadily across different time periods
- There are noticeable differences in transaction activity at the district level
- User adoption is growing over time, indicating increasing trust in digital payments
- Insurance-related transactions are present but relatively lower compared to regular transactions

These are the insights help in understanding how digital payments are spreading across different regions.

3. Visualizations Used

To make the data easier to understand, I used different types of visualizations in the dashboard.

- **Bar Charts:** Used to compare transaction data across states and districts

- **Line Charts:** Used to show trends over time (year-wise and quarter-wise growth)
- **Geographical Maps:** Used to visualize how transactions are distributed across India

All visualizations were created using Plotly and integrated into the Streamlit dashboard.

The dashboard also includes filters such as year, quarter, and state, allowing users to explore the data interactively.

Key Insights

From the analysis, the following insights were observed:

- States like Maharashtra and Karnataka show consistently high transaction volumes
- Digital payments have grown steadily across all quarters
- District-level analysis shows uneven distribution of transactions
- User adoption is increasing over time
- Insurance usage is present but lower compared to transactions

These insights can help businesses target high-growth regions.

Code Quality & Reusability

Modular Python scripts were used for data processing.

Separate functions for:

Data extraction, Transformation, Loading Clean and readable code structure, Reusable queries and components

Challenges Faced

While building this project, I faced several practical issues that don't usually show up in theory but come up during actual implementation.

1. Understanding the JSON Structure Properly

At the beginning, I assumed the JSON files would have a consistent structure, but that wasn't the case.

Some files had:

- a. Different nesting levels
- b. Missing fields in certain places
- c. Slight variations between transaction, user, and insurance data

I had to manually open multiple files and understand the structure before writing extraction code.

Initially, my code kept breaking because I was assuming fixed keys.

2. Debugging Data Extraction Errors

While extracting data using Python, I faced multiple errors like:

- **KeyErrors (when a key didn't exist)**
- **Empty values getting appended**
- **Wrong mapping of district/state data**

I had to print intermediate outputs and debug step-by-step to make sure the data was being extracted correctly.

3. Data Not Matching After Insertion into MySQL

After inserting data into MySQL, I noticed:

- **Some values were duplicated**
- **Some rows were missing**
- **Data types were mismatched (int vs float)**

I had to:

- a. **Drop tables and reload data multiple times**
- b. **Recheck my transformation logic**
- c. **Fix datatype issues manually**
- d. **Manipulating data where ever needed**

4. Writing Efficient SQL Queries

At first, my SQL queries were either:

- a. Too slow**
- b. Or giving incorrect aggregations**

Example issues:

- a. Wrong GROUP BY usage**
- b. Incorrect joins**
- c. Aggregation mistakes**

I improved this by testing queries separately in MySQL before using them in the dashboard.

5. Streamlit Not Updating Data Properly

While building the dashboard, one major issue was:

- Changing filters (state/year) didn't always update the output correctly

This happened due to:

- Improper state handling
- Queries not refreshing

I fixed this by restructuring how inputs and queries were connected.

6. Map Visualization Issues

While working with maps:

- Some states were not getting highlighted correctly
- State names in dataset didn't match map naming format

I had to manually align naming conventions to make the map work properly.

7. Performance Lag in Dashboard

When loading full data:

- **Dashboard became slow**
- **Charts took time to render**

I solved this by:

- Filtering data before visualization
- Avoiding unnecessary full dataset load

8. Managing Multiple Data Categories

Handling three different datasets:

- **Transactions**
- **Users**
- **Insurance**

Each had a different structure, so combining logic for all three in one dashboard was tricky.
Most important decisive task – **How I Solved Them?**

- Broke tasks into smaller steps for proper understanding taking it one step at a time
- Used Pandas for efficient transformation
- Tested each stage before moving forward

Conclusion

This project helped me understand the complete data analytics pipeline:

- **How to extraction of data in a proper way to ease the processing**
- **Cleaning and transforming it for processing**
- **Storing it in a database**
- **Visualizing insights through a dashboard for proper understanding**

It provided hands-on experience in working with real-world data.

Future Scope

The project can be improved further by:

- **Adding real-time data integration**

- **Applying Machine Learning models for prediction**
- **Detecting fraud patterns**
- **Enhancing dashboard UI/UX**
- **Expanding analysis to more granular data**

Project Links

Streamlit Dashboard: [Dashboard](#)

GitHub Repository: [GitHub](#)